

# Appendices

## Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>A</b> | <b>Pseudo code of algorithm</b>      | <b>1</b>  |
| <b>B</b> | <b>Framework setup</b>               | <b>2</b>  |
| <b>C</b> | <b>Real-world experiment setup</b>   | <b>9</b>  |
| <b>D</b> | <b>Simulation performance</b>        | <b>12</b> |
| <b>E</b> | <b>Build a house</b>                 | <b>16</b> |
| <b>F</b> | <b>Real-world demonstrations</b>     | <b>22</b> |
| <b>G</b> | <b>Additional empirical analysis</b> | <b>24</b> |

## A. Pseudo code of algorithm

---

**Algorithm 1** Interactive skill learning

---

```

1: Initialize Skill Library  $\mathcal{Z}_0$  and Examples  $\mathcal{E}_0$ 
2:  $z \leftarrow \text{SkillParser}(\text{skill\_description})$  {Choose current skill to learn}
3: while True do
4:    $l \leftarrow \text{task\_description}$  {Provide task instruction}
5:    $s_0 \leftarrow \text{TaskSetup}(\text{initial\_state\_description})$  {Set up the environment}
6:    $\text{correction} \leftarrow \emptyset$  {Set initial correction}
7:    $c \leftarrow \emptyset$  {Set initial task-specific code}
8:   while True do
9:      $\text{examples} \leftarrow (l'_i, c'_i)_{i=1, \dots, K} \in \mathcal{E}$  {Retrieval based on  $l$  and correction}
10:     $c, z \leftarrow \text{Agent}(l, c, z, \text{correction}, \text{examples})$ 
11:     $s_T \leftarrow \text{Rollout}(c)$  {Roll out policy code}
12:    if  $s_T$  is aligned with  $l$  then
13:       $\mathcal{E} \leftarrow \mathcal{E} \cup \{(l, c)\}$  {Add example to library}
14:      break
15:    end if
16:    update correction based on  $s_T$ 
17:  end while
18: end while
19: Update  $z$  in  $\mathcal{Z}$  {Update skill library}

```

---

## B. Framework setup

### Interactive terminal UI

To facilitate smooth and efficient interaction between the human user and the LLM-driven agent, we designed an interactive terminal-based user interface (UI). As illustrated in Fig. S1, the interface is divided into two components. The left panel displays the real-time rendering stream from the underlying simulation engine (e.g., PyBullet or MuJoCo). The right panel allows the user to communicate with the pipeline agent via keyboard input, either by selecting predefined options (such as learning a new skill or re-attempting previous tasks) or by providing free-form textual instructions or feedback.

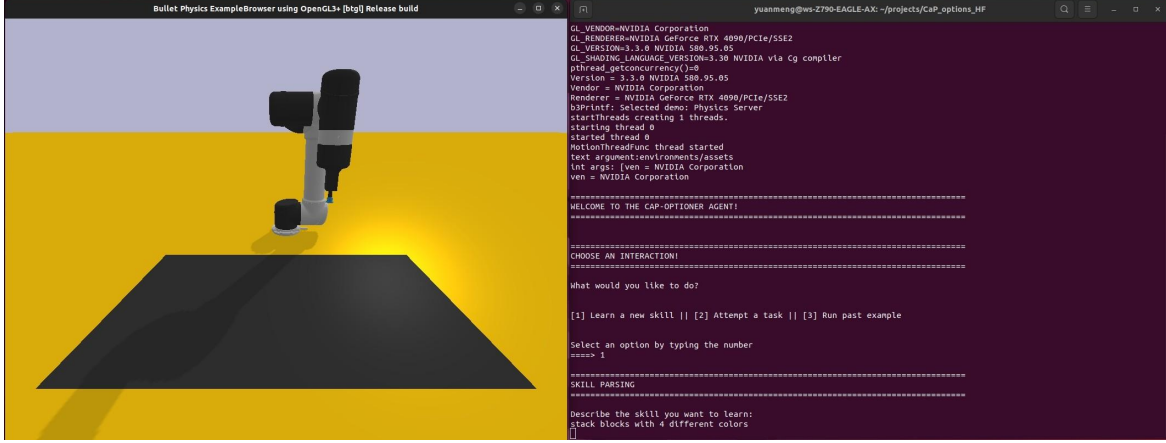


Figure S1. Snapshot of interactive terminal UI

### Framework prompting

Our framework and the baseline code generation methods rely on dynamic prompting for in-context learning. Table S1 lists the core primitives used in our framework, and the corresponding prompts are shown below.

Table S1. List of the core-primitives for our agent to build on

| Name                 | Function                                                                                                                                                                                                                                    |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| get_objects          | This function gets all objects in the environment. The agent can retrieve specific properties of these objects with the functions below.                                                                                                    |
| get_object_color     | Returns the color of the block.                                                                                                                                                                                                             |
| get_object_size      | Returns the size of the block.                                                                                                                                                                                                              |
| get_object_pose      | Returns the pose of the block, given as a 3-dimensional position vector, and a 4 dimensional quaternion rotation.                                                                                                                           |
| get_bbox             | Returns the axis-aligned bounding box of an object, to simplify collision queries.                                                                                                                                                          |
| put_first_on_second  | The main pick-and-place primitive. It picks up an object at the specified Pose, lifts it vertically to a specified height, moves along the x-y plane to a point directly above the place Pose, then moves it down until it detects contact. |
| move_end_effector_to | Moves the end effector the specified position, and suction gripper rotation.                                                                                                                                                                |

We use python format strings for our prompts.

*Listing 1. Main Actor prompts*

```
actor_system_prompt = f"""
You write python code to control a robotic arm in a simulated environment, building on an existing API.

You will be given:
- a task for the robotic agent to solve
- api functions you may use to solve the task
- if available, examples of codes that solve prior similar tasks

You are supposed to write flat code to solve the task, i.e. do not write any functions.
DO NOT make any imports.

Adhere to the following basic types:
{get_core_types_text()}
"""

def actor_prompt(task, few_shot_examples: list[TaskExample], api: list[Skill]):
    return f"""
    {get_few_shot_examples_string(few_shot_examples)}

    {get_skill_string(api)}
    The task is: {task}

    Write flat code to solve the task.
    """

def actor_iteration_prompt(feedback, examples: list[TaskExample] = []):
    return f"""
    Rewrite the previous code to integrate the feedback: {feedback}.
    {get_few_shot_examples_string(examples)}
    Only make changes that take into account this feedback.
    """
```

*Listing 2. Skill Learning prompts*

```
actor_skill_learning_system_prompt = f"""
You write python code to control a robotic arm in a simulated environment, building on an existing API.
We are trying to learn skills, and are using different tasks to test and effectively learn a specific skill.

You will be given:
- a task for the robotic agent to solve
- the skill you are supposed to use to solve the task

You are supposed to complete the function, as well as flat, task-specific code, as follows:

def given_function(...) -> ...:
    \"\"\" ... \"\"\"
    <function code>

<task-specific code>

For example:
-----
IN:
task: "put the red block on the green block"
skill:
def put_block_on_other_block(block: TaskObject, otherBlock: TaskObject):
    \"\"\" places the block on top of otherBlock \"\"\"
    pass

OUT:
def put_block_on_other_block(block: TaskObject, otherBlock: TaskObject):
    \"\"\" places the block on top of otherBlock \"\"\"
    put_first_on_second(get_object_pose(block), get_object_pose(otherBlock))

red_block = get_block(color="red")
green_block = get_block(color="green")
put_block_on_other_block(red_block, green_block)
-----

If the new task requires you to rewrite the function header, you may do so, for example,
to add arguments, or to update the docstring with important usage information.
You should try to preserve the previous functionality though, since the function might
have previously been used to solve other tasks, which should remain solvable after changes.

DO NOT make any imports.
DO NOT write any functions other than the given one.

Adhere to the following basic types:
{get_core_types_text()}
"""

def skill_learning_prompt(
    task,
    few_shot_examples: list[TaskExample],
```

```

skill: Skill,
other_useful_skills: list[Skill],
):
    return f"""
The task is: {task}
The function you are supposed to implement is:

{str(skill)}

-----

{get_few_shot_examples_string(few_shot_examples)}

The following skills may be useful in your implementation:
{"\\n\\n".join([skill.description for skill in other_useful_skills])}
-----

Implement the function and solve the task, while trying to ensure that prior tasks remain solvable.
"""

```

*Listing 3. Task Setup prompts*

```

task_setup_api_string = """
def add_block(
    self,
    env: Environment,
    color=None,
    size: tuple[float, float, float] = (0.04, 0.04, 0.04),
    pose: Pose=None
):
    "\\n\\n" adds a block of a given size and color to the environment
    If the pose is left unspecified, a random collision-free pose is selected
    "\\n\\n"

def add_zone(
    self,
    env: Environment,
    color: str,
    scale: float = 1,
    pose: Pose = None
):
    "\\n\\n" adds a zone of a given size and color to the environment
    If the pose is left unspecified, a random pose in the workspace is selected
    "\\n\\n"

def add_cylinder(self, env: Environment, color: str = "red", scale: float = 0.5):
    "\\n\\n" adds a cylinder of a given scale and color to the environment "\\n\\n"
"""

task_setup_system_prompt = f"""
You are writing python code to setup a simulated environment, translating user instructions
into executable code, based on an existing API.

You should adhere to the following types:
{get_core_types_text()}

You may use the following API:
{task_setup_api_string}

EXAMPLES:
#####

task: add 3 red blocks and 3 blue blocks
response:
for _ in range(3):
    self.add_block(env, "red")

for _ in range(3):
    self.add_block(env, "blue")

#####

task: add one big block and 4 blocks that are a quarter of the big blocks side length
response:
self.add_block(env, size=(0.08, 0.08, 0.08))
for _ in range(4):
    self.add_block(env, size=(0.02, 0.02, 0.02))

#####
"""

```

*Listing 4. Skill Parser prompts*

```

generate_function_header_system_prompt = f"""
We are working in the context of controlling a robotic arm with python code.
The user proposes a certain skill they would like the robot to learn.
To enable this, you are supposed to translate this skill into a python function,
i.e. choose a clear, descriptive name for the function, choose appropriate arguments,

```

```

and write a clear, descriptive docstring.

For example:
USER: "place one block on top of the other"
RESPONSE:
def place_block_on_other_block(block: TaskObject, otherBlock: TaskObject):
    \("\\" Places one block on top of the other block  \("\\"
    pass

Do not try to implement the function yet, that happens later.
You should adhere to the following types:
{get_core_types_text()}

The functions don't need Workspace as an argument, since there is only one.
"""

def generate_skill_prompt(prompt, similar_skills: list[Skill]):
    return f"""
    you may use the following function headers as examples of what you are trying to generate:
    {("\n".join([skill.description for skill in similar_skills]))}
    -----
    write a function header for the prompt: {prompt}.
    """

def refine_function_header_prompt(function_code, refinement):
    return f"""
    Your role is to refine an existing python function, for example by adding a function argument or
    changing the name. If the function is implemented (i.e. not just "pass"), you should also alter
    the implementation accordingly, making as little changes and assumptions as possible.
    Revise the following python function according to the user instructions:
    {function_code}
    Refinement prompt:
    {refinement}
    Do not make any assumptions.
    """

class ParsedList(BaseModel):
    parsed_list: list[str]

def parse_hint_to_list_prompt(hint):
    return f"""
    The user provided a list of tasks that are similar to the one you are currently
    trying to solve, in a single string. Retrieve each of the task descriptions from
    this string and return them as a list.
    This is the string: {hint}
    """

```

*Listing 5. Core primitives*

```

"""
-----
the following functions require an initialised environment -
the agent doesn't need to know anything about the environment, only what methods are available to it
we are responsible for properly initialising the environment, and ensuring that the agent has access to it
"""

"""
IMPORTANT
- pybullet can only handle one server at a time, if this is not commented out, this is the environment being used
"""

# env = Environment(
#     "/Users/maxfest/vscode/thesis/thesis/environments/assets",
#     disp=True,
#     shared_memory=False,
#     hz=480,
#     record_cfg={
#         "save_video": False,
#         "save_video_path": "${data_dir}/${task}-cap/videos/",
#         "add_text": True,
#         "add_task_text": True,
#         "fps": 20,
#         "video_height": 640,
#         "video_width": 720,
#     },
# )
# from tasks.tasks.place_blocks import Place5Blocks

# task = Place5Blocks()

# env.set_task(task)
# env.reset()
"""-----"""

__all__ = [
    "get_objects",
    "get_object_size",
    "get_object_pose",
    "get_object_color",
    "get_end_effector_pose",
    "put_first_on_second",

```

```

        "move_end_effector_to",
        "get_bbox",
        "get_point_at_distance_and_rotation_from_point",
    ]

def get_objects() -> list[TaskObject]:
    """gets all objects in the environment"""
    return env.task.taskObjects

def get_object_size(task_object: TaskObject) -> tuple[float, float, float]:
    """Returns the size of the given TaskObject as a tuple (width, depth, height)."""
    return task_object.size

def get_object_pose(obj: TaskObject) -> Pose:
    """returns the pose (Point3D, Rotation) of a given object in the environment."""
    return _from_pybullet_pose(env.get_object_pose(obj.id))

def get_object_color(task_object: TaskObject) -> str:
    return task_object.color

def get_end_effector_pose() -> Pose:
    """gets the current pose of the end effector"""
    return _from_pybullet_pose(env.get_ee_pose())

def put_first_on_second(pickPose: Pose, placePose: Pose):
    """
    This is the main pick-and-place primitive.
    It allows you to pick up the TaskObject at 'pickPose', and place it at the Pose specified by 'placePose'.
    If 'placePose' is occupied, it places the object on top of 'placePose'.
    """
    return env.step(
        action={
            "pose0": _to_pybullet_pose(pickPose),
            "pose1": _to_pybullet_pose(placePose),
        }
    )

def move_end_effector_to(pose: Pose, speed=0.001):
    """moves the end effector from its current Pose to a given new Pose"""
    env.movep(_to_pybullet_pose(pose), speed=speed)

def get_bbox(obj: TaskObject) -> AABBBoundingBox:
    """gets the axis-aligned bounding box of an object - this is useful primarily for collision detection"""
    aabb_min, aabb_max = env.get_bounding_box(obj.id)
    return AABBBoundingBox(Point3D.from_xyz(aabb_min), Point3D.from_xyz(aabb_max))

def get_point_at_distance_and_rotation_from_point(
    point: Point3D, rotation: Rotation, distance: float, direction=np.array([1, 0, 0])
) -> Point3D:
    """compute a point that is at a specific 'distance' from 'point', at a specified 'rotation'
    The direction specifies the base direction in which to apply the rotation.
    This is useful for placing objects relative to other objects.
    """
    rotated_direction = rotation.apply(direction)
    new_point = point.np_vec + distance * rotated_direction
    return Point3D.from_xyz(new_point)
    
```

## Baseline model setup

To ensure a fair comparison, we leverage state-of-the-art open-source code and language generation repositories. We adapt the baseline implementations and provide wrappers for direct integration with our APIs:

- **CaP:** [https://github.com/google-research/google-research/tree/master/code\\_as\\_policies](https://github.com/google-research/google-research/tree/master/code_as_policies)
- **LoHoRavens:** <https://github.com/Shengqiang-Zhang/LoHo-Ravens>
- **DAHLIA:** <https://github.com/Ghiara/DAHLIA>

We also consider Voyager (Wang et al., 2023), which employs LLMs for automatic skill learning and RAG retrieval in Minecraft. However, Voyager is built for a Java-based game environment, and adapting it to our Python-based robotic control is impractical. To enable comparison, we design a variation of our framework where LLMs provide feedback (**LYRA (Ours) w/ LLM feedback**), keeping close to Voyager’s concept. Following prior works (Zhang et al., 2023b; Meng et al., 2025b; Wang et al., 2023), the LLM evaluates task success by receiving RGB-D observations before and after execution and providing corrective feedback if needed. In practice, we find that letting the LLM directly discover missing capabilities and learn new skills is difficult, since robotic manipulation involves more complex dynamics and spatial reasoning than Minecraft. Therefore, in this variation, we do not attempt direct skill learning but instead rely on the learned skills and

example databases from our framework. The LLM focuses on reasoning about the current task, providing feedback to adjust the plan, and retrieving relevant items from memory.

For ablation, we also test a version without the retrieval module (**LYRA (Ours) w/o memory**). Here, learned skills and examples are randomly sampled from our databases and appended to the prompt until the context window is filled. This simulates catastrophic forgetting in approaches that only modify prompts with human-in-the-loop (Arenas et al., 2024). Specifically, we include 25 out of 49 learned skills and 25 out of 86 examples for Ravens tasks. This setup allows us to study how learned skills contribute when recent works rely solely on examples for in-context learning. Although performance drops on more complex tasks due to limited access to the full skill library, this variation still outperforms baseline models.

We further conducted a hint-specific ablation on three Ravens tasks of increasing complexity: “stack blocks”, “make a human face”, and “build a house”. We retained the previously learned sub-skills, but removed the final task-specific skill and examples. For **LYRA w/o hint**, an external volunteer unfamiliar with the skill library provided only task-level feedback, while the agent relied on standard similarity-based retrieval. After obtaining one successful solution, we evaluated robustness over 50 randomized trials using five initialization seeds. As shown in Fig. S2, hints have little effect on the simple “stack blocks” task (0.96 vs. 0.98), where similarity-based retrieval already identifies the relevant skills. For more complex tasks, removing hints reduces success from 0.90 to 0.74 for “make a human face” and from 0.84 to 0.70 for “build a house”. In these cases, retrieval may omit semantically less obvious but necessary sub-skills, such as `identify_beam_block`. Although human feedback can partially compensate by generating flat code, such code is often hard-coded and less robust to randomized initial scenes. These results indicate that hints are unnecessary for simple tasks but improve retrieval efficiency in more complex ultra-long-horizon tasks by reusing acquired skills instead of reconstructing them.

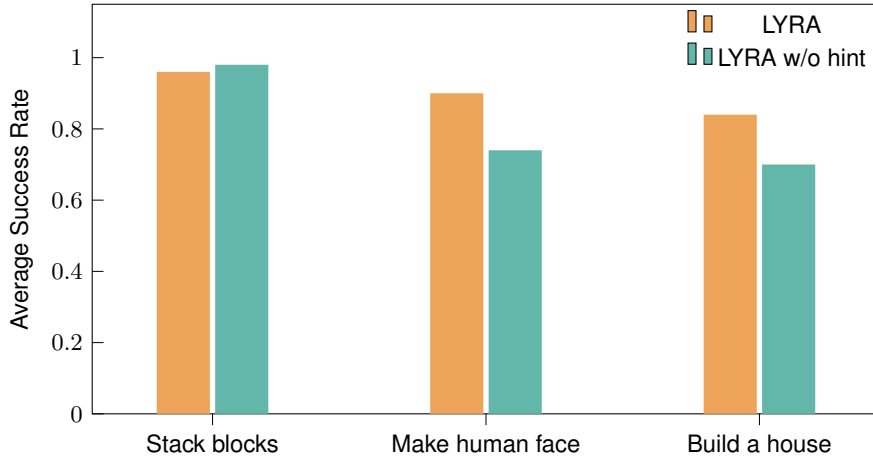


Figure S2. Ablation study of LYRA without hint application for Raven three tasks. We conduct 50 trials with the initialization-state reset.

Since both the baseline models and our framework rely on few-shot in-context learning for reasoning and task planning, we configure the few-shot examples and/or skills for the baselines as follows:

- **Customized Ravens:** All models, including ours, start with 9 core primitives as initial skills, which are also appended to the baselines for in-context learning. For few-shot adaptation, all baselines are provided with the following example task plans and are expected to generalize to unseen scenarios, either in an open-loop manner or under LLM guidance:
  - stack blocks (4 colors)
  - put the yellow block next to the green block
  - build a  $\{2 \times 2 \times 2\}$  cube with 8 blocks
  - build a  $\{3 \times 2 \times 2\}$  pyramid with 8 blocks
  - put the red block in the middle of the workspace
  - rotate the blue block by 45 degrees
  - move the end effector to the center of the workspace
  - move the smallest block 10cm to the left

- arrange the blocks around a circle
- **Franka Kitchen & Metaworld & LIBERO-long:** All models, including ours, start with 9 core primitives as initial skills (shared across both benchmarks via unified API wrappers). For few-shot adaptation, baselines are given the following example task plans and are expected to generalize to new scenarios:
  - reach a goal
  - push a puck to a goal
  - pick and place a puck in a goal
  - open a door (revolute joint)
  - close a window (slide joint)
  - open an oven button (revolute joint)

### Raven tasks setup

In the Experiment section, we compare the success rates across 6 customized Ravens tasks as a case study. The tasks are set up as follows:

- **Place block next to reference:** A scene with two large blocks (red and blue). The agent must place the red block next to the blue one without collision. Alignment details (e.g., corner-to-corner) are not required for success. This task helps the agent acquire foundational spatial skills.
- **Stack blocks:** A scene with four medium-sized colored blocks. The agent must stack all blocks into a tower. Precise alignment is not required for success evaluation; towers with random positions or orientations are also considered successful. This provides a base skill for more complex behaviors.
- **Build  $\{i \times j \times k\}$  structure:** A scene with multiple medium-sized colored blocks. The agent must build structures as instructed by the user. Two example plans are provided in the prompt: (1) a  $\{2 \times 2 \times 2\}$  cube and (2) a  $\{3 \times 2 \times 2\}$  pyramid. The agent must also generalize to unseen tasks: (3) a  $\{4 \times 3 \times 3\}$  pyramid, (4) a  $\{1 \times 3 \times 3\}$  wall, and (5) a  $\{4 \times 4 \times 1\}$  base.
- **Make a human face:** A scene with several medium-sized colored blocks and one rectangular block. The agent must build a circle and use two blocks plus the rectangle to form facial features inside. Orientation details are not required for success.
- **Build a Jenga tower:** A scene with multiple rectangular blocks of uniform or mixed colors. The agent must stack them in alternating orientations to construct a stable Jenga tower.
- **Build a house:** A scene with blocks of various colors and shapes. The agent must use yellow blocks for the house base, brown blocks for the roof base, and red plates for the tiles.

## C. Real-world experiment setup

### Hardware setting

In the real-world demonstrations, we employ a Franka FR3 manipulator paired with an Intel RealSense D435i depth camera mounted at the table edge to evaluate the proposed framework. As illustrated in Fig. S3, the camera is positioned approximately 1 m in front of the robot, closely mirroring the configuration used in simulation. The robot’s base coordinate system,  $O_{base}$ , is defined at the center of joint 0, with the x-axis directed toward the table and the z-axis aligned vertically upward from the table surface. For object manipulation, the robot is equipped with a Franka two-finger gripper, in contrast to the simulation benchmark, which employs a suction gripper for pick-and-place tasks. To ensure seamless deployment across embodiments and gripper types, we maintain consistent primitive APIs between the real-world and simulated settings, enabling skills developed in simulation to transfer directly to the physical robot.

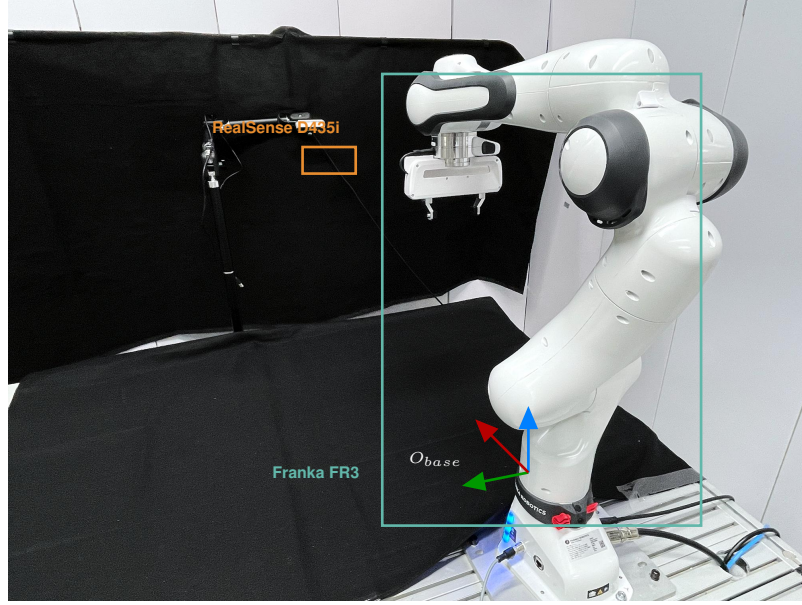


Figure S3. Real-world hardware setup. A Franka FR3 serves as the agent embodiment, with an Intel RealSense D435i placed at the table edge to capture privileged scene information.

In our real-world setup, the calibrated extrinsic transformation between the camera and the robot base frame is obtained as:

$$T_{base}^{cam} = \begin{bmatrix} 6.12323400e-17 & 7.37277337e-01 & -6.75590208e-01 & 1.06 \\ 1.00000000e+00 & -4.51452165e-17 & 4.13679693e-17 & 0.16 \\ -0.00000000e+00 & -6.75590208e-01 & -7.37277337e-01 & 0.61 \\ 0. & 0. & 0. & 1.00 \end{bmatrix} \quad (1)$$

### Perception

In simulation, privileged object information (such as type, size, pose, and color) can be directly accessed through predefined APIs, e.g., `get_objects`, `get_object_size`, and `get_object_pose`. In contrast, acquiring such information in the real world is more challenging. To address this, we employ recent vision foundation models to perceive and parse the task scene. For open-world object pose estimation, we use Grounded SAM 2 (Ren et al., 2024) to detect target object masks and their bounding boxes (with SAM 2.1.hiera.large and GroundingDINO.swint.ogc). The object center is estimated by computing the mean of all mask pixels and projecting it back into 3D space. For orientation estimation, we adopt FoundationPose (Wen et al., 2024) in combination with object CAD models to recover the object’s coordinate frame, particularly the x- and y-axis directions. By comparing the angle differences between the object and robot base coordinate frames, we determine the object’s rotation pose.

## Deployment

We utilize the official Franka ROS2 library on Ubuntu 22.04 to control the Franka FR3 manipulator. The action space of our framework is defined in a 6-dimensional vector space, representing end-effector motions composed of linear displacements along the x, y, and z axes, as well as rotational changes in roll, pitch, and yaw. By default, Franka ROS2 relies on OMPL-based motion planning. To better align with our simulation environment and enable more intuitive trajectory control, we extend this setup by customizing a Cartesian linear motion planning instance, which allows for straightforward and precise linear path execution. To ensure reliable operation in the real-world hardware setup, we carefully configure the planning parameters, including detection tolerances, velocity and acceleration scaling factors, and orientation/pose constraints. The complete parameter configuration for our motion planning setup is summarized in Table S2, which provides the exact values adopted in our experiments.

Table S2. Hardware parameter setup

| Parameters                                   | Values                |
|----------------------------------------------|-----------------------|
| RealSense x-axis detection tolerance         | 0.01 [m]              |
| RealSense y-axis detection tolerance         | 0.01 [m]              |
| RealSense z-axis detection tolerance         | 0.01 [m]              |
| Feasible factor of cartesian linear planning | 0.9                   |
| Max velocity scaling factor                  | 0.1                   |
| Max acceleration scaling factor              | 0.1                   |
| Absolute x-axis orientation tolerance        | 0.02                  |
| Absolute y-axis orientation tolerance        | 0.02                  |
| Absolute z-axis orientation tolerance        | 0.02                  |
| Target pose constraint in x-axis             | 0.005 [m]             |
| Target pose constraint in y-axis             | 0.005 [m]             |
| Target pose constraint in z-axis             | 0.005 [m]             |
| Workspace in x-axis                          | +0.25 [m] - +0.8 [m]  |
| Workspace in y-axis                          | -0.55 [m] - +0.3 [m]  |
| Workspace in z-axis                          | +0.01 [m] - +0.65 [m] |

The following is an example prompt for generating a task-specific code plan that can be deployed in the ROS2 environment.

Listing 6. Demo task script structure for ROS2 deployment

```
from utils.core_types import Workspace
# The task script should be summarized as follows to enable a direct deployment in ROS2 environment:

def demo(record_cfg, camera_args):
    # -----
    # Do not modify this part, because all task demo are execute under ROS2 environment node
    rclpy.init()
    executor = MultiThreadedExecutor()
    env = RealWorldEnvironment(record_cfg=rec_cfg, camera_args=camera_args)
    executor.add_node(env)
    executor_thread = threading.Thread(target=executor.spin, daemon=True)
    executor_thread.start()
    env.reset()
    # -----
    try:
        # ... You can implement your task specific code at here ...
        # ... flat code ...
        # ... call your skill_function(...) to accomplish the task ...

    # -----
    # You donot need to touch this part
```

```
except KeyboardInterrupt:
    # env.end_rec()
    print("\n Test interrupted by user")
except Exception as e:
    # env.end_rec()
    print(f"\n Test failed: {str(e)}")
    import traceback
    traceback.print_exc()
finally:
    try:
        env.destroy_node()
        # env.end_rec()
    except:
        pass
    executor.shutdown()
    rclpy.shutdown()
# -----
```

## D. Simulation performance

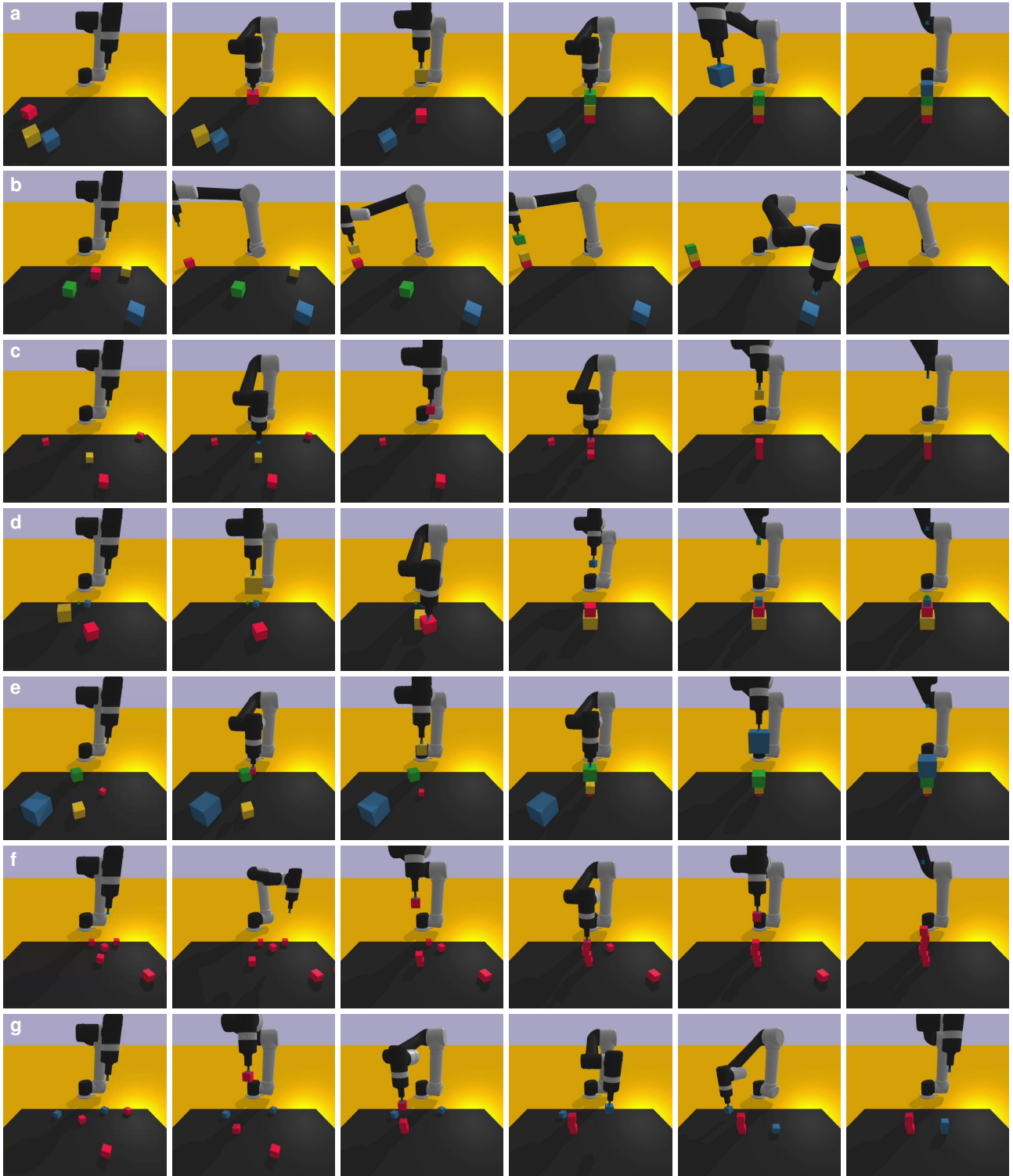


Figure S4. Snapshots of the “stack blocks” task and its variations under user-guided lifelong learning. (a) Stack 4 blocks corner-to-corner at the workspace center. (b) Stack 4 blocks with a  $45^\circ$  rotation at the back-right table corner. (c) Stack blocks by color, with the yellow block on top. (d) Stack blocks from largest to smallest. (e) Stack blocks from smallest to largest. (f) Stack blocks into a zigzag tower. (g) Build two towers sorted by color.

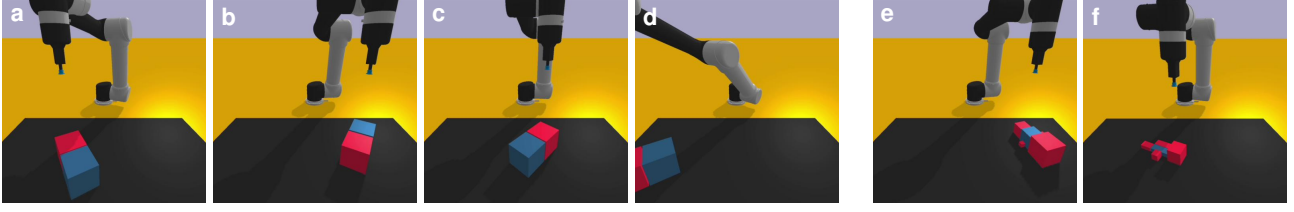


Figure S5. Snapshots of the task “place block next to reference.” (a)–(d) Basic skill that places a red block next to a blue block with a 0.5 cm gap along different axes. (e)–(f) Extended capability through user-guided lifelong learning, where the agent places red blocks on all sides of the blue block to form a cross.



Figure S6. Snapshots of the task “build  $\{i * j * k\}$  {structure}”. (a) Build a  $2 \times 2 \times 2$  cube. (b) Build a  $3 \times 2 \times 2$  pyramid. (c) Build a  $4 \times 3 \times 3$  pyramid. (d) Build a  $1 \times 3 \times 3$  wall. (e) Build a  $4 \times 4 \times 1$  base.

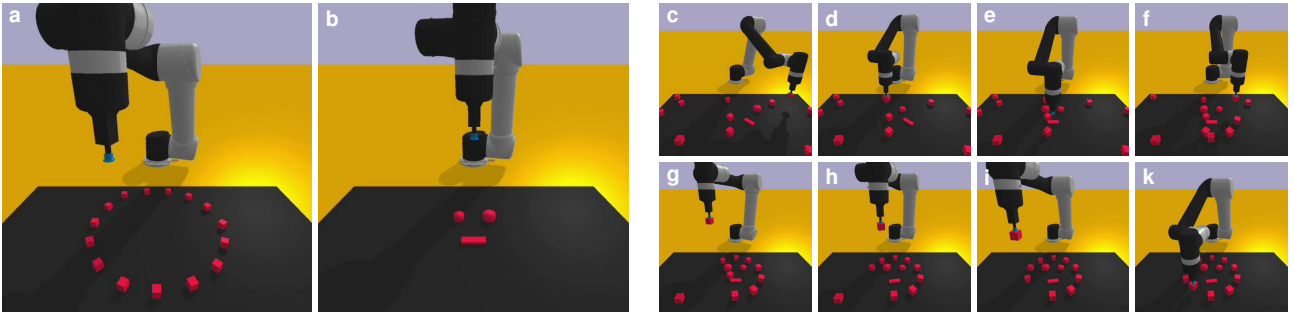


Figure S7. Snapshots of the “make smiley human face” task. (a)–(b) Basic skills “arrange\_in\_circle” and “make\_face\_feature,” used to construct the circular outline and the eyes and mouth with blocks. (c)–(k) Extended capability through user-guided lifelong learning, where the agent recalls and reuses these basic skills to complete the full human face structure.

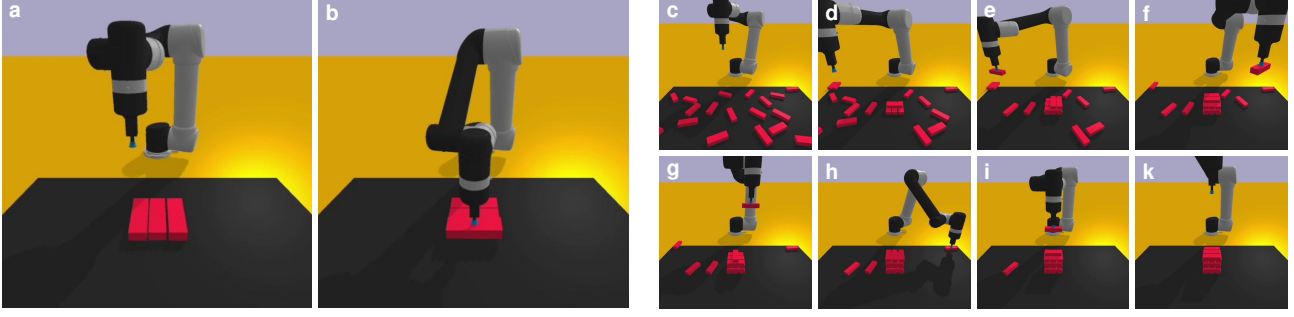


Figure S8. Snapshots of the “build jenga tower” task. (a)–(b) Basic skill “build\_jenga\_layer,” which constructs a single Jenga layer in the required orientation. (c)–(k) Extended capability through lifelong learning, where the agent reuses skills to stack layers and complete the Jenga tower.

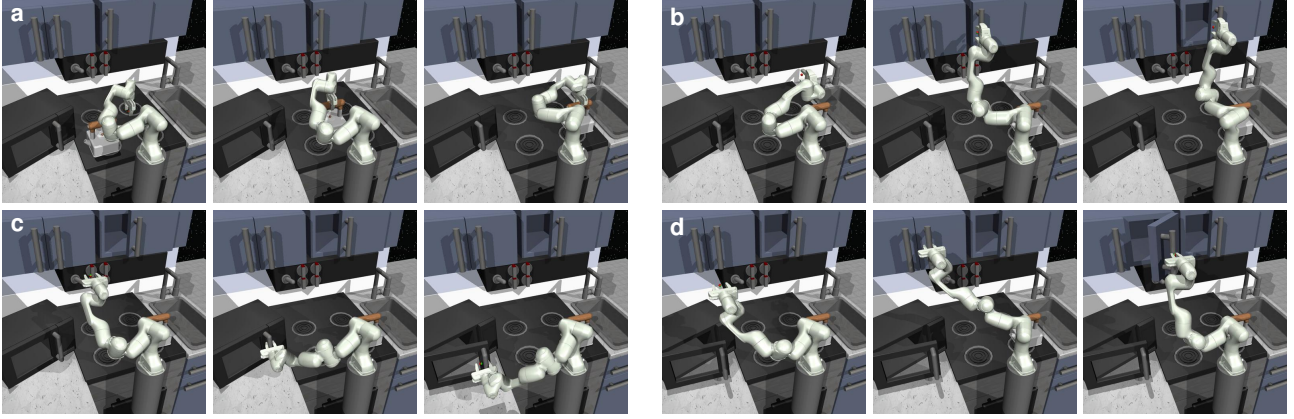


Figure S9. Snapshots of Franka Kitchen. (a) pick-place kettle; (b) open slide-cabinet; (c) open microwave; (d) open hinge-cabinet.

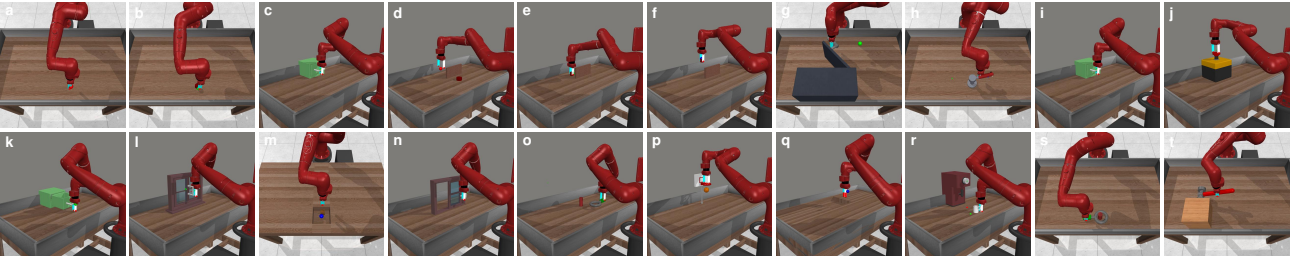


Figure S10. Snapshots of Meta-World. (a) Reach; (b) Push; (c) Pick-place; (d) Reach-wall; (e) Push-wall; (f) Pick-place-wall; (g) Door-open; (h) Faucet-open; (i) Drawer-close; (j) Button-press; (k) Drawer-open; (l) Window-close; (m) Sweep-into; (n) Window-open; (o) Disassembly; (p) Basketball; (q) Pick-out-of-hole; (r) Coffee-pull; (s) Assembly; (t) Hammer.

Table S3. Number of Corrections ( $\downarrow$ ) for individual tasks in Franka Kitchen (FK) and MetaWorld (MW). “-” means failed in the task.

| Framework              | FK - kettle     | FK - slide cabinet   | FK - hinge cabinet | FK - microwave   | MW - reach            | MW - push         | MW - pick-place  | MW - reach-wall   |
|------------------------|-----------------|----------------------|--------------------|------------------|-----------------------|-------------------|------------------|-------------------|
| DAHLIA                 | 6               | 3                    | 5                  | 5                | 0                     | 3                 | 6                | 7                 |
| LYRA (Ours) w/o memory | 4               | 1                    | 8                  | 7                | 0                     | 9                 | 6                | 2                 |
| LYRA (Ours)            | 2               | 2                    | 4                  | 3                | 0                     | 5                 | 5                | 3                 |
|                        | MW - push-wall  | MW - pick-place-wall | MW - door-open     | MW - faucet-open | MW - drawer-close     | MW - button-press | MW - drawer-open | MW - window-close |
| DAHLIA                 | 6               | -                    | 2                  | 3                | 2                     | 4                 | 3                | 2                 |
| LYRA (Ours) w/o memory | 6               | 7                    | 1                  | 4                | 3                     | 4                 | 3                | 4                 |
| LYRA (Ours)            | 4               | 3                    | 0                  | 3                | 1                     | 1                 | 4                | 2                 |
|                        | MW - sweep-into | MW - window-open     | MW - disassembly   | MW - basketball  | MW - pick-out-of-hole | MW - coffee-pull  | MW - assembly    | MW - hammer       |
| DAHLIA                 | 7               | 2                    | 4                  | 3                | 4                     | -                 | -                | -                 |
| LYRA (Ours) w/o memory | 7               | 3                    | 2                  | 1                | 5                     | 3                 | 9                | 4                 |
| LYRA (Ours)            | 3               | 0                    | 1                  | 1                | 3                     | 2                 | 4                | 6                 |

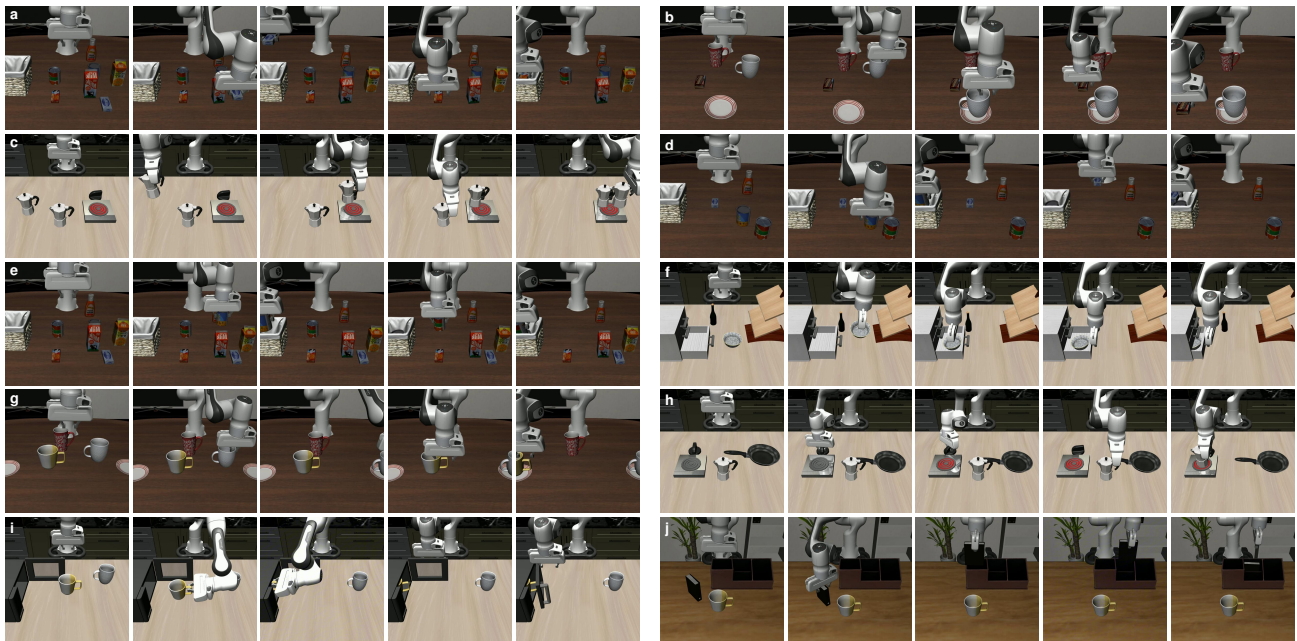


Figure S11. Snapshots of our framework performance in LIBERO-long tasks. (a) - (j) LIBERO-long tasks that require the agent to manipulate multiple objects in a common scene.

## E. Build a house

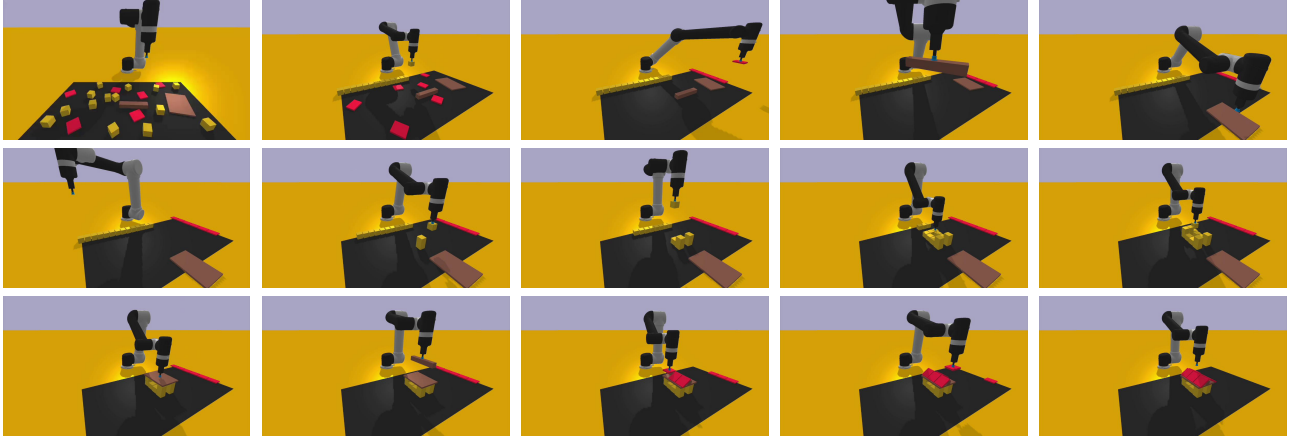


Figure S12. Snapshots of “build a house”.

The implementation details of the skill “build a house” are demonstrated as follows:

Listing 7. Function build\_house

```
from utils.core_types import Workspace

def build_house():
    """Builds a house in the middle of the workspace.
    Assumes all the necessary objects are available in the workspace,
    and moves them out of the way before building the house.
    """
    objects = get_blocks_by_color()
    base_blocks = get_blocks_by_color("yellow")
    if len(base_blocks) != 14:
        raise Exception("Not enough blocks to build the house")
    roof_base = identify_roof_base(objects)
    if not roof_base:
        raise Exception("Can't find the roof base")
    roof_beam = identify_beam_block(objects)
    if not roof_beam:
        raise Exception("Can't find the roof beam")
    roof_tiles = identify_roof_tiles(objects)
    if not roof_tiles:
        raise Exception("Can't find the roof tiles")

    # Move objects out of the way
    # Move base blocks to back edge of workspace
    back_left = Workspace.back_left
    make_line_with_blocks(
        base_blocks, Pose(back_left, Rotation.from_euler("z", np.pi / 2))
    )

    # Move roof tiles to right side of workspace
    back_right = Workspace.back_right
    make_line_with_blocks(roof_tiles, Pose(back_right, Rotation.identity()))

    # Move roof beam to front edge of workspace
    front_left = Workspace.front_left
    put_first_on_second(
        get_object_pose(roof_beam), Pose(front_left, Rotation.identity())
    )

    # Move roof base to middle of the front edge
    front_middle = Point3D(Workspace.front_left.x, Workspace.middle.y, 0)
    put_first_on_second(
        get_object_pose(roof_base), Pose(front_middle, Rotation.identity())
    )

    # Build the house
    middle = Workspace.middle
    build_house_base(base_blocks, Pose(middle, Rotation.identity()))

    assemble_roof(roof_base, roof_beam, roof_tiles, Pose(middle, Rotation.identity()))
```

Listing 8. Function build\_house\_base

```
def build_house_base(blocks: list[TaskObject], pose: Pose):
    """Constructs the base of a house in the workspace using a list of block
    TaskObjects starting from a specified pose.
    Args:
```

```

blocks (list[TaskObject]): A list of blocks to use in forming the base of a house.
Each block should have uniform attributes such as size and color.
startingPose (Pose): The starting position and orientation in the workspace from
which to begin constructing the base of the house.
"""

block_width = get_object_size(blocks[0])[0]
startingPose = Pose(
    pose.position.translate(Point3D(1.5 * block_width, -block_width, 0)),
    pose.rotation,
)

# make the front left corner of the house
stack_blocks(blocks[0:2], startingPose)

# leave a gap of one block width
next_stack_position = get_point_at_distance_and_rotation_from_point(
    startingPose.position,
    startingPose.rotation,
    block_width * 2,
    direction=(0, 1, 0),
)
stack_blocks(blocks[2:4], Pose(next_stack_position, startingPose.rotation))

# make lines towards the back of the workspace from each of the stacks
make_line_of_blocks_next_to(blocks[4:6], blocks[0], "back")
make_line_of_blocks_next_to(blocks[6:8], blocks[3], "back")

# make the back wall of the house
back_wall_start_pos = get_point_at_distance_and_rotation_from_point(
    get_object_pose(blocks[5]).position,
    startingPose.rotation,
    block_width + 0.005,
    direction=(-1, 0, 0),
)

build_structure_from_blocks(
    blocks[8:14], (1, 3, 2), Pose(back_wall_start_pos, startingPose.rotation)
)

```

*Listing 9. Function identify\_roof\_tiles*

```

def identify_roof_tiles(objects: list[TaskObject]) -> list[TaskObject]:
    """Identifies and returns a list of TaskObjects that are categorized as roof tiles
    from a given list of objects. A roof tile is characterized by having one dimension smaller
    than 0.02 and being red in color.
    Args:
    - objects (list[TaskObject]): A list of TaskObjects to be analyzed for identification of roof tiles.
    Returns:
    - list[TaskObject]: A list of TaskObjects that are identified as roof tiles,
    based on the specified characteristics.
    """
    roof_tiles = []

    for obj in objects:
        if obj.color.lower() == "red" and any(dim < 0.02 for dim in obj.size):
            roof_tiles.append(obj)
    return roof_tiles

```

*Listing 10. Function identify\_beam\_block*

```

def identify_beam_block(blocks: list[TaskObject]) -> TaskObject:
    """Identifies the beam block from a list of blocks.
    A beam block is defined by the following criteria:
    - It must have the color 'brown'.
    - It has one square side, meaning two side lengths must be the same.
    - The third side should be at least 3 times as long as the square sides.
    Args:
    blocks (list[TaskObject]): The list of block objects to be evaluated.
    Returns:
    TaskObject: Returns the TaskObject identified as a beam block.
    If no beam block is found, returns None.
    """

    for block in blocks:
        if block.color != "brown":
            continue
        width, depth, height = sorted(block.size)
        if width == depth and height >= 3 * width:
            return block
    return None

```

*Listing 11. Function identify\_roof\_base*

```

def identify_roof_base(objects: list[TaskObject]) -> TaskObject:
    """ Identifies and returns the TaskObject that serves as the base for a roof in a given list of objects.
    A roof base is characterized by being brown in color and by having two dimensions that are at least

```

```

10 times larger than the third dimension.
Args:
- objects (list[TaskObject]): A list of TaskObjects to be analyzed for identification of the roof base.
Returns:
- TaskObject: The TaskObject identified as the roof base, based on the specified characteristics.
"""
for obj in objects:
    size = obj.size
    if obj.color == 'brown': # Changed from 'red' to 'brown'
        dimensions = sorted(size)
        if dimensions[0] * 10 <= dimensions[1] and dimensions[0] * 10 <= dimensions[2]:
            return obj
return None

```

*Listing 12. Function get\_blocks\_by\_color*

```

def get_blocks_by_color(color: str = None) -> list[TaskObject]:
    """
    Retrieves all block objects in the workspace.
    If a specific color is provided, only blocks of that color are retrieved.
    Args:
    color (str, optional): The color of the blocks to retrieve.
                        If not specified, retrieves all blocks regardless of their color.
    Returns:
    list[TaskObject]: A list of TaskObject instances representing the blocks in the workspace.
    """
    all_objects = get_objects()
    if color:
        return [obj for obj in all_objects if obj.objectType == 'block' and obj.color == color]
    else:
        return [obj for obj in all_objects if obj.objectType == 'block']

```

*Listing 13. Function assemble\_roof*

```

def assemble_roof(base: TaskObject, roof_beam: TaskObject,
                  roof_tiles: list[TaskObject], overall_pose: Pose):
    """
    Assembles a roof structure using a designated base, a roof beam, and a list of roof tiles,
    starting from a given overall pose. The base acts as the foundation while the roof beam provides
    structural support and the roof tiles are placed on top to complete the structure.
    Args:
    - base (TaskObject): The TaskObject representing the base upon which the roof is built.
    - roof_beam (TaskObject): The TaskObject representing the beam supporting the roof tiles
    between the base and the tiles.
    - roof_tiles (list[TaskObject]): A list of TaskObjects representing the roof tiles to
    be placed on the beam.
    - overall_pose (Pose): The Pose indicating the overall position and orientation for the roof assembly.
    """
    # Place base in the middle of the workspace
    put_first_on_second(get_object_pose(base), overall_pose)
    # Compute the pose for the roof beam
    base_pose = get_object_pose(base)
    beam_pose = Pose(
        Point3D(
            base_pose.position.x,
            base_pose.position.y,
            base_pose.position.z + (base.size[2] / 2) + (roof_beam.size[2] / 2)
        ),
        base_pose.rotation
    )
    # Place beam on the base
    put_first_on_second(get_object_pose(roof_beam), beam_pose)
    # Compute the pose for the roof tiles on top of the beam
    beam_pose = get_object_pose(roof_beam)
    roof_tiles_pose = Pose(
        Point3D(
            beam_pose.position.x,
            beam_pose.position.y,
            beam_pose.position.z + roof_beam.size[2] / 2 + 0.01
        ),
        beam_pose.rotation
    )
    # Place roof tiles
    place_roof_tiles(roof_tiles, roof_tiles_pose)

```

*Listing 14. Function build\_structure\_from\_blocks*

```

from utils.core_types import *

def build_structure_from_blocks(
    blocks: list[TaskObject],
    dimensions: tuple[int, int, int],
    pose: Pose,
    gap: float = 0.005,
):
    """
    Assembles a structure using individual block TaskObjects based on the specified dimensions

```

```

and the given pose. The blocks should be positioned to form the desired structure starting
from the given pose, which specifies the position and orientation of the first block placed.
Assumes that the list 'blocks' contains enough block TaskObjects to construct the specified
structure. Assumes the blocks are homogeneous in size.
Arranges the blocks to form a 3D structure of the given dimensions.
"""
if len(dimensions) != 3:
    raise ValueError("Dimensions should be a tuple of three integers.")

block_size = get_object_size(blocks[0])
block_index = 0

for z in range(dimensions[2]):
    layer_blocks = blocks[block_index : block_index + dimensions[0] * dimensions[1]]
    layer_start_position = get_point_at_distance_and_rotation_from_point(
        pose.position,
        pose.rotation,
        (block_size[2] + gap) * z,
        direction=np.array([0, 0, 1]),
    )
    layer_start_pose = Pose(layer_start_position, pose.rotation)
    for y in range(dimensions[1]):
        row_blocks = layer_blocks[y * dimensions[0] : (y + 1) * dimensions[0]]
        row_start_position = get_point_at_distance_and_rotation_from_point(
            layer_start_pose.position,
            layer_start_pose.rotation,
            (block_size[1] + gap) * y,
            direction=np.array([0, 1, 0]),
        )
        row_start_pose = Pose(row_start_position, layer_start_pose.rotation)
        make_line_with_blocks(row_blocks, row_start_pose, gap=gap)
        block_index += dimensions[0] * dimensions[1]

```

*Listing 15. Function place\_roof\_tiles*

```

def place_roof_tiles(roof_tiles: list[TaskObject], specific_pose: Pose):
    """
    Places exactly six roof tiles starting from a specific pose.
    Arranges the roof tiles evenly from the specified starting position and orientation.
    Ensures that the list of roof tiles has exactly six elements before proceeding.
    Args:
    - roof_tiles (list[TaskObject]): A list of TaskObjects identified as roof tiles.
    Must contain exactly six tiles.
    - specific_pose (Pose): The Pose representing the starting position
    and orientation for tile placement.
    """
    if len(roof_tiles) != 6:
        raise ValueError("There must be exactly six roof tiles")
    tile_width = roof_tiles[0].size[0]

    # Rotate the tiles by 90 degrees relative to the starting pose
    relative_rotation = specific_pose.rotation * Rotation.from_euler('z', 90, degrees=True)
    adjusted_pose_left = Pose(
        Point3D(specific_pose.position.x, specific_pose.position.y-tile_width/2, specific_pose.position.z),
        relative_rotation
    )
    put_first_on_second(get_object_pose(roof_tiles[0]), adjusted_pose_left)
    # Place one block on either side of the first block on the left
    move_block_next_to_reference(roof_tiles[1], roof_tiles[0], axis='-y', gap=0.005)
    move_block_next_to_reference(roof_tiles[2], roof_tiles[0], axis='y', gap=0.005)
    # Place the first block of the second set to the right (x direction) of the first block of the first set
    move_block_next_to_reference(roof_tiles[3], roof_tiles[0], axis='x', gap=0.005)
    # Place one block on either side of the first block of the second set
    move_block_next_to_reference(roof_tiles[4], roof_tiles[3], axis='-y', gap=0.005)
    move_block_next_to_reference(roof_tiles[5], roof_tiles[3], axis='y', gap=0.005)

```

*Listing 16. Function make\_line\_of\_blocks\_next\_to*

```

def make_line_of_blocks_next_to(blocks: list[TaskObject], referenceBlock: TaskObject,
                                direction: str, gap: float = 0.005):
    """
    Arranges the given blocks in a straight line next to a reference block in the
    specified direction.
    Args:
    blocks (list[TaskObject]): A list of TaskObject instances representing
    the blocks to be arranged in a line.
    referenceBlock (TaskObject): The TaskObject representing the reference
    block next to which the line will start.
    direction (str): A string indicating the direction in which to align the line of blocks.
    Valid directions are "front", "back", "left", and "right".
    gap (float): The gap between the reference block and the first block in the line,
    and between consecutive blocks.
    This function will arrange the specified blocks in a single line, starting from the chosen
    side of the reference block, following the given direction along the x or y axis in the workspace,
    depending on the specified direction.
    """
    axis = ''
    if direction == "front":
        axis = 'x'

```

```

elif direction == "back":
    axis = '-x'
elif direction == "left":
    axis = '-y'
elif direction == "right":
    axis = 'y'
else:
    raise ValueError("Invalid direction provided. Use 'front', 'back', 'left', or 'right'.")
current_reference = referenceBlock
for block in blocks:
    move_block_next_to_reference(block, current_reference, axis=axis, gap=gap)
    current_reference = block

```

*Listing 17. Function make\_line\_with\_blocks*

```

from utils.core_types import TaskObject, Pose
from utils.core_primitives import (
    get_object_pose,
    get_object_size,
    put_first_on_second,
    get_point_at_distance_and_rotation_from_point,
)

def make_line_with_blocks(
    blocks: list[TaskObject], start_pose: Pose, gap: float = 0.005
):
    """Arranges the given blocks in a straight line starting from the specified start pose.
    Args:
        blocks (list[TaskObject]): A list of block objects to be arranged in a line.
        start_pose (Pose): The pose in the workspace where the line of blocks should start.
            The position will be used as the starting point, and the rotation
            will be used as the direction vector.
        gap (float): The gap between consecutive blocks.
    Note:
        The function places the blocks in the order in which they are passed.
    """
    current_block = blocks[0]
    put_first_on_second(get_object_pose(current_block), start_pose)
    for block in blocks[1:]:
        # Get the current pose of the block
        move_block_next_to_reference(block, current_block, axis="x", gap=gap)
        current_block = block

```

*Listing 18. Function stack\_blocks*

```

def stack_blocks(blocks: list[TaskObject], start_pose: Pose):
    """Stacks a sequence of blocks on top of each other starting from a specified pose.
    Args:
        blocks: A list of TaskObject instances representing the blocks to be stacked.
        start_pose: A Pose indicating the initial position and orientation of the bottom block.
    """
    # Check if there's already a block in the starting position
    all_blocks = get_objects()
    for block in all_blocks:
        block_pose = get_object_pose(block)
        # Check if block is close enough to the starting position, considering some tolerance
        if (
            abs(block_pose.position.x - start_pose.position.x) < 0.05 and
            abs(block_pose.position.y - start_pose.position.y) < 0.05 and
            abs(block_pose.position.z - start_pose.position.z) < 0.05
        ):
            # Move the block to the top-left corner (back_left)
            workspace = Workspace()
            top_left_position = workspace.back_left
            put_first_on_second(block_pose, Pose(top_left_position, Rotation.identity()))
            break

    current_pose = start_pose
    for block in blocks:
        pick_pose = get_object_pose(block) # Get current pose of the block
        put_first_on_second(pick_pose, current_pose) # Place block on the current pose
        # Get the size of the block to calculate the new position for the next block
        block_size = get_object_size(block)
        # Update the current pose to place the next block on top
        current_pose = Pose(
            position=Point3D(
                x=current_pose.position.x,
                y=current_pose.position.y,
                z=current_pose.position.z
                + block_size[2], # Increment z by block height
            ),
            rotation=current_pose.rotation,
        )

```

*Listing 19. Function move\_block\_next\_to\_reference*

```

def move_block_next_to_reference(

```

```

    block: TaskObject, referenceBlock: TaskObject, axis: str = "x", gap: float = 0.005
):
    """Moves the block next to the referenceBlock such that their edges are aligned along
    the specified axis with a small gap.
    Args:
        block (TaskObject): The block object to be moved and aligned.
        referenceBlock (TaskObject): The block object that remains stationary and serves as the reference.
        axis (str): The axis along which to align the blocks. Should be 'x', '-x', 'y', or '-y'.
        gap (float, optional): The small gap to leave between the blocks. Defaults to 0.005 meters.
    Raises:
        ValueError: If the specified axis is not 'x', '-x', 'y', or '-y'.
    """
    if axis not in ["x", "-x", "y", "-y"]:
        raise ValueError("Axis must be either 'x', '-x', 'y', or '-y'.")
    # Get the pose and size of the blocks
    block_pose = get_object_pose(block)
    reference_pose = get_object_pose(referenceBlock)
    reference_size = get_object_size(referenceBlock)
    block_size = get_object_size(block)
    # Determine the offset distance based on the axis
    if axis == "x":
        offset = (reference_size[0] + block_size[0]) / 2 + gap
        direction = np.array([1, 0, 0]) # positive x-axis direction
    elif axis == "-x":
        offset = (reference_size[0] + block_size[0]) / 2 + gap
        direction = np.array([-1, 0, 0]) # negative x-axis direction
    elif axis == "y":
        offset = (reference_size[1] + block_size[1]) / 2 + gap
        direction = np.array([0, 1, 0]) # positive y-axis direction
    elif axis == "-y":
        offset = (reference_size[1] + block_size[1]) / 2 + gap
        direction = np.array([0, -1, 0]) # negative y-axis direction

    rotated_direction = reference_pose.rotation.apply(direction)
    new_position = reference_pose.position.numpy_vec + offset * rotated_direction
    new_position = Point3D.from_xyz(new_position)
    # New pose for the block with the same rotation as the reference
    new_pose = Pose(position=new_position, rotation=reference_pose.rotation)
    # Move the block
    put_first_on_second(block_pose, new_pose)

```

Listing 20. Applied core primitives

```

def get_object_pose(obj: TaskObject) -> Pose:
    """Returns the pose (Point3D, Rotation) of a given object in the environment."""
    return _from_pybullet_pose(env.get_object_pose(obj.id))

def put_first_on_second(pickPose: Pose, placePose: Pose):
    """
    This is the main pick-and-place primitive.
    It allows you to pick up the TaskObject at 'pickPose', and place it at the Pose specified by 'placePose'.
    If 'placePose' is occupied, it places the object on top of 'placePose'.
    """
    return env.step(
        action={
            "pose0": _to_pybullet_pose(pickPose),
            "pose1": _to_pybullet_pose(placePose),
        }
    )

def get_object_size(task_object: TaskObject) -> tuple[float, float, float]:
    """Returns the size of the given TaskObject as a tuple (width, depth, height)."""
    return task_object.size

def get_object_color(task_object: TaskObject) -> str:
    """Returns the color of the given TaskObject."""
    return task_object.color

def get_objects() -> list[TaskObject]:
    """gets all objects in the environment"""
    return env.task.taskObjects

```

## F. Real-world demonstrations

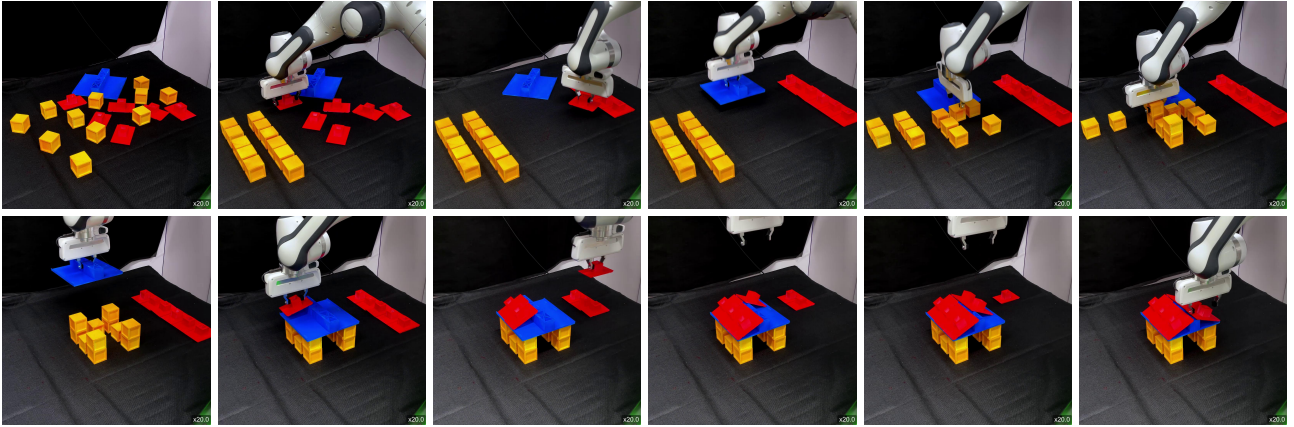


Figure S13. Real-world demonstration for task “build a house”.

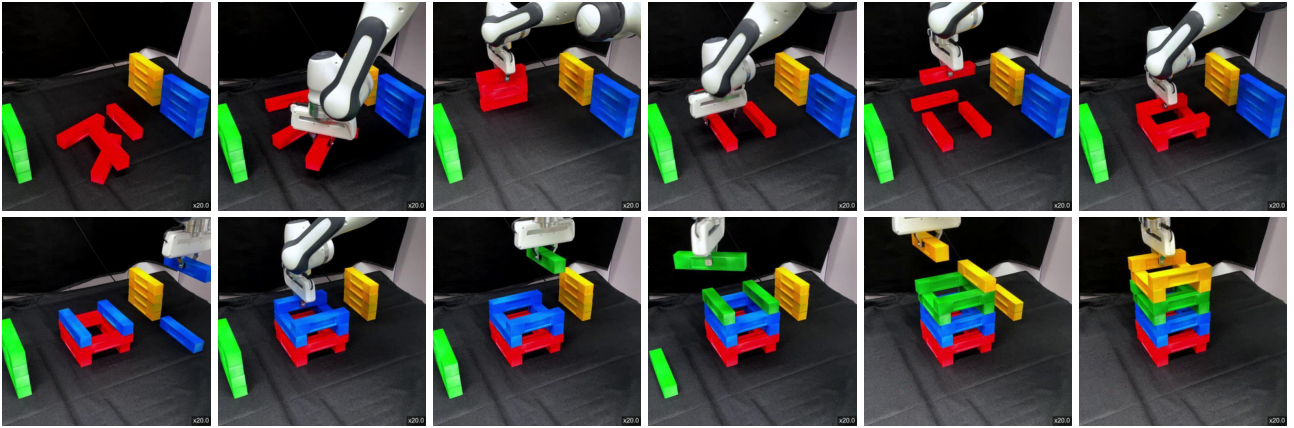


Figure S14. Real-world demonstration for task “stack a jenga tower”.

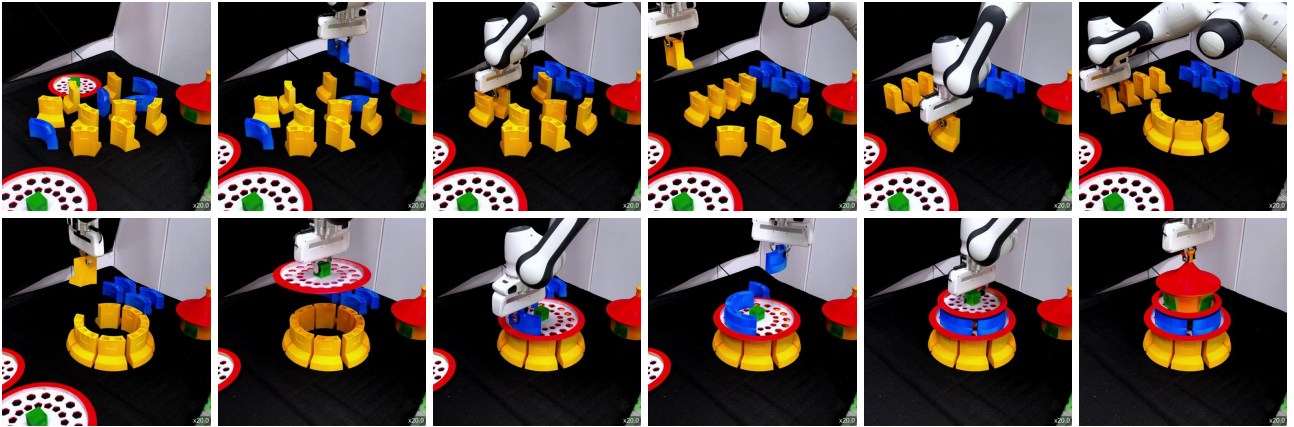


Figure S15. Real-world demonstration for task “build a temple”.

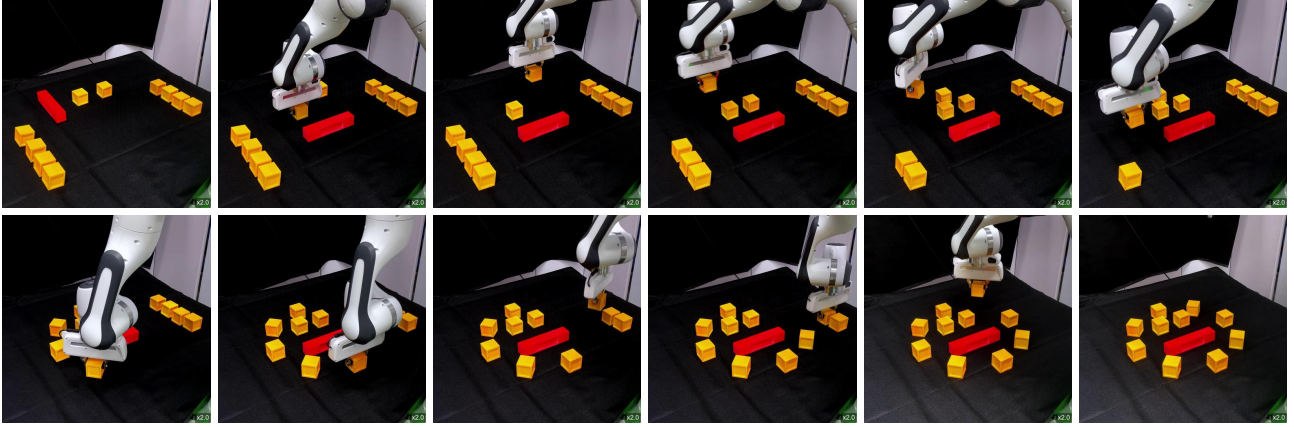


Figure S16. Real-world demonstration for task “build a human face”.

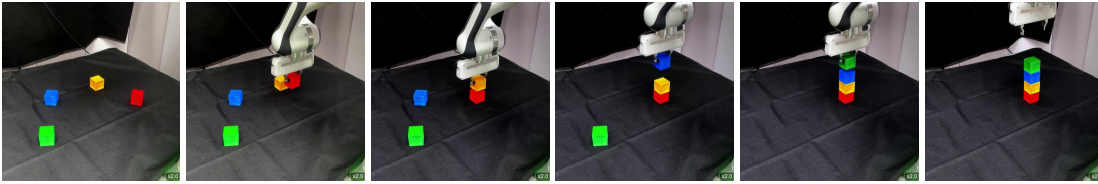


Figure S17. Real-world demonstration for task “stack blocks” with corner-to-corner aligned.

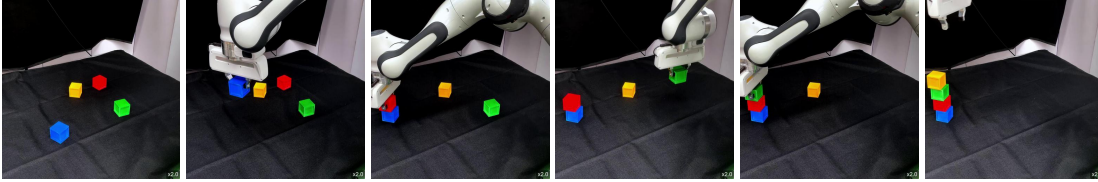


Figure S18. Real-world demonstration for task “stack blocks” with shift 0.5cm from each other.

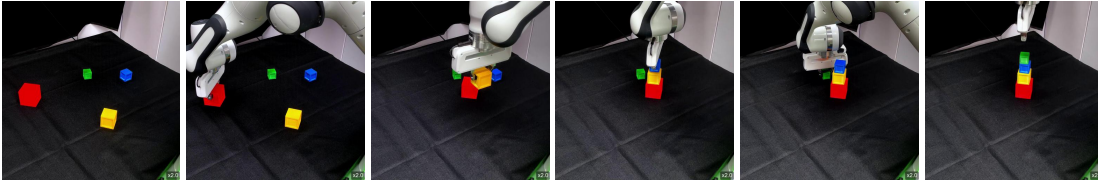


Figure S19. Real-world demonstration for task “stack blocks” from big to small.

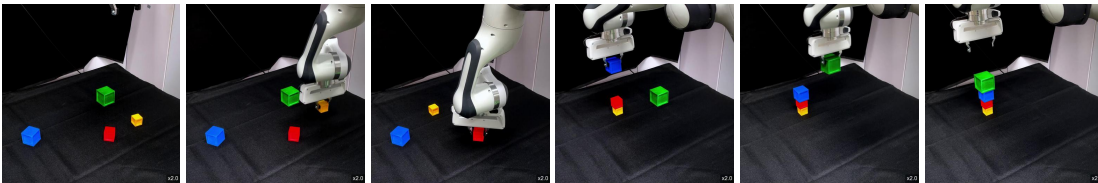


Figure S20. Real-world demonstration for task “stack blocks” from small to big.

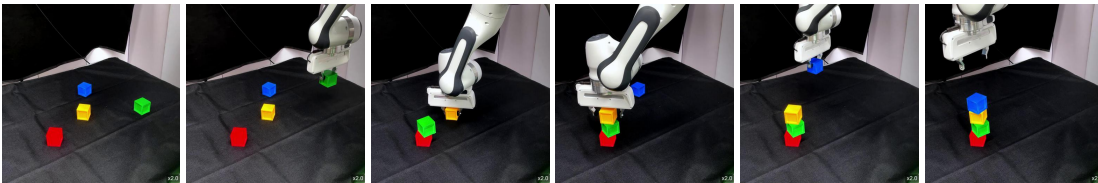


Figure S21. Real-world demonstration of the “stack blocks” task, showcasing the performance of the baseline models, e.g., CaP or DAHLIA.

## G. Additional empirical analysis

### Baseline comparison

Table S4 presents the empirical comparison between three representative hierarchical and lifelong RL baselines and our proposed framework LYRA across 10 Meta-World manipulation tasks. For RL baselines, each method is trained for 1 million steps per task, and evaluated using five random seeds, reporting average success rates. In our experiment, we include the following baselines:

- **HiLL** (Zhang et al., 2023a): A hierarchical RL framework that leverages a latent landmark graph to balance RL exploration and exploitation.
- **Text2Reward** (Xie et al., 2024): An LLM-guided reward refinement framework that generates dense reward trajectories for RL.
- **LEGION** (Meng et al., 2025a): A state-of-the-art lifelong RL framework that leverages a Bayesian Non-parametric model for dynamic knowledge preservation and adaptation.

For HiLL and Text2Reward, which are designed for standard single-task RL, we report their fine-tuned performance on each task independently. For the lifelong RL baseline LEGION, we report the best performance achieved across its asynchronous learning process. We additionally evaluate “Forgetting” as defined in (Meng et al., 2025a), where the scalar ranges from 0 to 1 (0 = no forgetting, 1 = complete forgetting). For our framework LYRA, we report performance under two settings:  $\leq 3$  number of corrections (NoC) and  $\leq 5$  NoC.

The RL baselines exhibit mixed performance across individual tasks, with average success rates of 0.64 (HiLL), 0.78 (Text2Reward), and 0.72 (LEGION) after 1M training steps per task. In contrast, LYRA achieves an average success rate of 0.80 using no more than three human corrections, and reaches a 1.0 success rate on all tasks with at most five corrections. For relatively simple primitives such as Reach, Door-open, and Button-press, LYRA achieves perfect success even without any human intervention, indicating that the learned skill library generalizes robustly to these scenarios. We also evaluate knowledge preservation using the Forgetting metric defined in LEGION, where lower values indicate better retention. LYRA achieves a perfect score of 0.00, comparable to LEGION’s 0.02, demonstrating that its externally stored skill representations avoid the catastrophic forgetting common in lifelong RL. While the RL methods perform reasonably well on short-horizon, single-primitive tasks, they remain fundamentally limited in long-horizon reasoning and fail entirely on multi-stage Ravens tasks such as Build a House. These results highlight the efficiency and robustness of LYRA’s correction-driven, skill-based paradigm and underscore the difficulty current RL systems face in scaling to extreme long-horizon manipulation.

Table S4. Comparison of hierarchical and lifelong RL baselines and LYRA on Meta-World tasks, including success rates and forgetting.

| Method               | Success rate( $\uparrow$ ) |      |            |           |             |              |              |             |              |        |               | Forgetting( $\downarrow$ ) |  |
|----------------------|----------------------------|------|------------|-----------|-------------|--------------|--------------|-------------|--------------|--------|---------------|----------------------------|--|
|                      | Reach                      | Push | Sweep-into | Door-open | Faucet-open | Drawer-close | Button-press | Coffee-pull | Window-close | Hammer | Total average |                            |  |
| HiLL                 | 1.0                        | 0.6  | 0.4        | 0.8       | 0.8         | 0.8          | 1.0          | 0.2         | 0.8          | 0.0    | 0.64          | N/A                        |  |
| Text2Reward          | 1.0                        | 0.6  | 0.6        | 1.0       | 1.0         | 0.8          | 1.0          | 0.6         | 0.8          | 0.4    | 0.78          | N/A                        |  |
| LEGION               | 0.8                        | 1.0  | 0.8        | 0.6       | 1.0         | 0.6          | 0.8          | 0.4         | 1.0          | 0.2    | 0.72          | 0.02                       |  |
| LYRA ( $\leq 3$ NoC) | 1.0                        | 0.0  | 1.0        | 1.0       | 1.0         | 1.0          | 1.0          | 1.0         | 1.0          | 0.0    | 0.80          | 0.00                       |  |
| LYRA ( $\leq 5$ NoC) | 1.0                        | 1.0  | 1.0        | 1.0       | 1.0         | 1.0          | 1.0          | 1.0         | 1.0          | 1.0    | 1.00          | 0.00                       |  |

Table S5 reports the comparison between two recent large-scale VLA models (OpenVLA and  $\pi_0$ ) and our proposed framework across short- and long-horizon manipulation benchmarks. Specifically, we evaluated all methods on five Meta-World (MW) tasks to validate short-horizon performance, and further compare them on LIBERO-long as well as our customized extreme long-horizon Ravens tasks (e.g., building a jenga tower, constructing a  $4 \times 4 \times 3$  pyramid, and building a house). For consistency, in Metaworld and our Ravens tasks, we collected 30 trajectories per task using our skill-based code generation framework, recording synchronized third-person views and wrist-camera frames ( $224 \times 224$  px). The collection and data preprocessing costs are approximately 5 hours. We used joint-space actions for all VLA models. Both OpenVLA and  $\pi_0$  were fine-tuned for 50k steps using LoRA on their officially released LIBERO-pretrained weights, following the original hyperparameters.

As shown in the table, on the five short-horizon Meta-World tasks, both OpenVLA and  $\pi_0$  achieve performance comparable to LYRA, indicating that pretrained VLA policies are reliable in simple single-primitive settings. However, their performance

degrades substantially when evaluated on long-horizon tasks. Even on LIBERO-long, where each task requires only two sequential primitives, OpenVLA and  $\pi_0$  reach success rates of only 0.65 and 0.60, respectively. On our customized Ravens benchmarks, which require over 20 primitives and significantly richer spatial reasoning, both models fail entirely, achieving 0.0 success on tasks such as build a jenga tower, construct a  $4 \times 4 \times 3$  pyramid, and build a house. In contrast, LYRA successfully completes all tasks using its library of compositional and reusable skills, reaching success rates between 0.80 and 0.95 across these extreme long-horizon scenarios for task plan generation. We attribute the VLA failures to two factors: (1) data sparsity, as VLA models require hundreds of expert demonstrations even for two-primitive tasks and still exhibit limited generalization; and (2) restricted distributional diversity in existing VLA training benchmarks, which often assume narrow object initialization ranges. By comparison, our Ravens environments sample object placements across the full workspace, making visual and spatial generalization more demanding. These findings highlight the limitations of current large-scale VLA models in extreme long-horizon settings and demonstrate the advantages of LYRA’s skill-based, correction-efficient approach for robust compositional manipulation.

Table S5. Comparison between OpenVLA,  $\pi_0$ , and LYRA on Meta-World, LIBERO-long, and long-horizon Ravens tasks.

|             | MW – Pick-place | MW – Door-open | MW – Window-close | MW – Coffee-pull | MW – Hammer | LIBERO – long | Build a Jenga tower | Construct a $4 \times 4 \times 3$ pyramid | Build a House |
|-------------|-----------------|----------------|-------------------|------------------|-------------|---------------|---------------------|-------------------------------------------|---------------|
| OpenVLA     | 0.95            | 0.90           | 1.00              | 0.95             | 0.85        | 0.65          | 0.00                | 0.00                                      | 0.00          |
| $\pi_0$     | 0.90            | 1.00           | 1.00              | 0.90             | 0.90        | 0.60          | 0.00                | 0.00                                      | 0.00          |
| LYRA (Ours) | 1.00            | 1.00           | 1.00              | 1.00             | 1.00        | 0.95          | 0.95                | 0.85                                      | 0.80          |

### Time efficiency comparison

To quantify the time cost associated with human-in-the-loop feedback, we analyze timestamped logs from 90 feedback loops collected in the Ravens benchmark, across three human users (each provides 30 data logs). Our interactive terminal UI allows users to input free-form text in real time, enabling precise measurement of the full human feedback cycle. Because the interface provides predefined interaction options, we use three timestamps as reference points: (1)  $t_1$ , the moment the task execution finishes; (2)  $t_2$ , the moment the user selects the “Provide Feedback” option; and (3)  $t_3$ , the moment the feedback is submitted. The interval  $t_2 - t_1$  represents the user’s reasoning and decision-making time, and  $t_3 - t_2$  represents the time required to type and submit the feedback. Table S6 reports the measured approximate time cost across the full feedback loop, including the decision-making phase, and the text-entry or text-generation phase. A complete human feedback cycle, including both reasoning and typing, takes on average 27.6 seconds, while an LLM produces feedback in 17.6 seconds, with overlapping minimum and maximum ranges. When isolating the actual decision-making time, humans are consistently faster, requiring less than 8 seconds to determine the correct correction, whereas LLMs require more than 14 seconds for inference and reasoning. The additional time on the human side stems primarily from keyboard typing (over 75% of total time), which reflects typing speed rather than cognitive load. These results indicate that the human cognitive burden during feedback is low, and the time difference between human and LLM feedback is incremental rather than fundamental. Although LLMs generate text faster, their feedback must adhere to structured formats in prior works (Meng et al., 2025b; Zhang et al., 2023b; Zhi et al., 2025), limiting expressiveness and generalization. In contrast, our use of free-form human feedback provides far richer, more adaptive guidance, which we find essential for enabling the agent to escape local optima and acquire robust, reusable skills in long-horizon settings.

Table S6. Human-in-the-loop vs. LLM response time analysis.

|                                         | Human user                            | LLMs                               |
|-----------------------------------------|---------------------------------------|------------------------------------|
| Average time of a feedback loop         | 27.6 sec (Min 11.2 sec, Max 39.5 sec) | 17.6 sec (Min 4 sec, Max 32.4 sec) |
| Average time for reasoning/decision     | 7.5 sec (below 25%)                   | 14.2 sec (over 90%)                |
| Average time for text typing/generation | 20.1 sec (over 75%)                   | 3.4 sec (below 10%)                |